FICHA TÉCNICA DE PROPUESTA DE INVESTIGACIÓN — TEMA 1

Título: Implementación de una arquitectura multiplataforma unificada con Kotlin Multiplatform (KMP) y Compose for Web (Kotlin/Wasm) para la optimización del Time-to-Interactive (TTI) e Interaction to Next Paint (INP) en sistemas de telemetría industrial, 2026

Línea UTP: Ingeniería de Software y Sistemas de Información | **Diseño:** Aplicada, cuantitativa, experimental de banco de pruebas (Benchmark frente a React/SPA tradicional)

Vigencia Bibliográfica: 2021–2026 (≤ 5 años) | **Anti-Plagio SUNEDU:** Totalmente inédito en RENATI/SUNEDU. Evalúa Compose for Web compilado a WebAssembly (Wasm) nativo compartiendo el 100% de la lógica y componentes UI con la app Android.

1. REALIDAD PROBLEMÁTICA Y OBJETIVOS

Problema General: ¿De qué manera una arquitectura unificada con KMP y Compose WebAssembly (Wasm) influye en la reducción del Time-to-Interactive y el uso de memoria respecto al desarrollo web JavaScript convencional?

Objetivo General: Demostrar que una arquitectura en KMP con compilación a WebAssembly optimiza significativamente el Time-to-Interactive (TTI) y el consumo de CPU/RAM en plataformas web y móviles unificadas.

Objetivos Específicos:
• Medir el rendimiento basal (TTI, FCP y métrica INP) de la aplicación web tradicional con Chrome DevTools Profiler (Pre-test).
• Construir la solución multiplataforma compartiendo la capa de dominio, red (Ktor) y presentación con Compose Multiplatform (Kotlin/Wasm).
• Comparar el rendimiento de ejecución, reducción de deuda técnica y tasa de reutilización de código (Post-test).

2. MATRIZ DE OPERACIONALIZACIÓN DE VARIABLES

Variable	Nombre de la Variable	Dimensiones	Indicadores Medibles
VI (Causa)	Arquitectura multiplataforma con KMP y Compose Web (Kotlin/Wasm)	Compilación a código binario WebAssembly, Unificación de código multiplataforma	Porcentaje de código compartido (% LOC), tiempo de compilación Wasm (s), tamaño del bundle descargado (MB).
VD (Efecto)	Rendimiento web y eficiencia de ejecución del cliente de telemetría	Métricas Core Web Vitals, Eficiencia de recursos del navegador	Time-to-Interactive - TTI (ms), Interaction to Next Paint - INP (ms), uso de memoria Heap en navegador (MB).

3. REFERENCIAS TÉCNICAS Y CIENTÍFICAS VIGENTES (2021–2026)

• **[VI]** Kotlin Multiplatform Docs (2024) — https://kotlinlang.org/docs/multiplatform.html
• **[VI]** Compose for Web (Wasm) (2024) — https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform-wasm.html
• **[VD]** Interaction to Next Paint (INP) (2024) — https://web.dev/articles/inp
• **[VD]** Google Core Web Vitals Guide (2024) — https://web.dev/articles/vitals

4. JUSTIFICACIÓN TÉCNICA DE INGENIERÍA

Compose for Web compilado a Wasm ejecuta directamente instrucciones en la máquina virtual WebAssembly sin la sobrecarga del recolector de basura de JS, logrando fluidez de 60 fps en navegadores y compartiendo el 90% de código con Android.